

PYTHON

Python is an interpreted, object-oriented programming language similar to PERL, that has gained popularity because of its clear syntax and readability

Note: Python is case Sensitive

WHY PYTHON ?

- Python works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc.)
- Python has a simple syntax similar to the English language.
- Python has syntax that allows developers to write programs with fewer lines than some other programming languages.
- Python runs on an interpreter system, meaning that code can be executed as soon as it is written. This means that prototyping can be very quick.
- Python can be treated in a procedural way, an object-orientated way or a functional way.

PYTHON APPLICATIONS

- web development (server-side) - django
- software development - tkinter
- Mathematics - scipy
- Machine Learning - numpy, pandas
- web scraping - beautifulsoup
- Astropy
- Earthpy
- Boto 3

LEVELS OF LANGUAGE

High level Language

C(1972), C++(1985), Python(1991), Java(1996)

Assembly level Language

Micro Processor Language – 8085,8086

Low level Language

Machine Language – 1011 0001

HIGH LEVEL LANGUAGE

C,C++ - Compiler Python - Interpreter

Java – Interpreter/Compiler

Compiler

It converts instructions into a machine-code or lower-level form so that they can be read and executed by a computer.

Interpreter

It can analyze and execute a program line by line.

HISTORY OF PYTHON

Founder

Guido van Rossum at Netherland

Version history

- Python 1x - 1991
- Python 2x - (2000 - Jan 1, 2020)
- Python 3x (Py3K) - 2008

ASSEMBLY LEVEL & LOW LEVEL

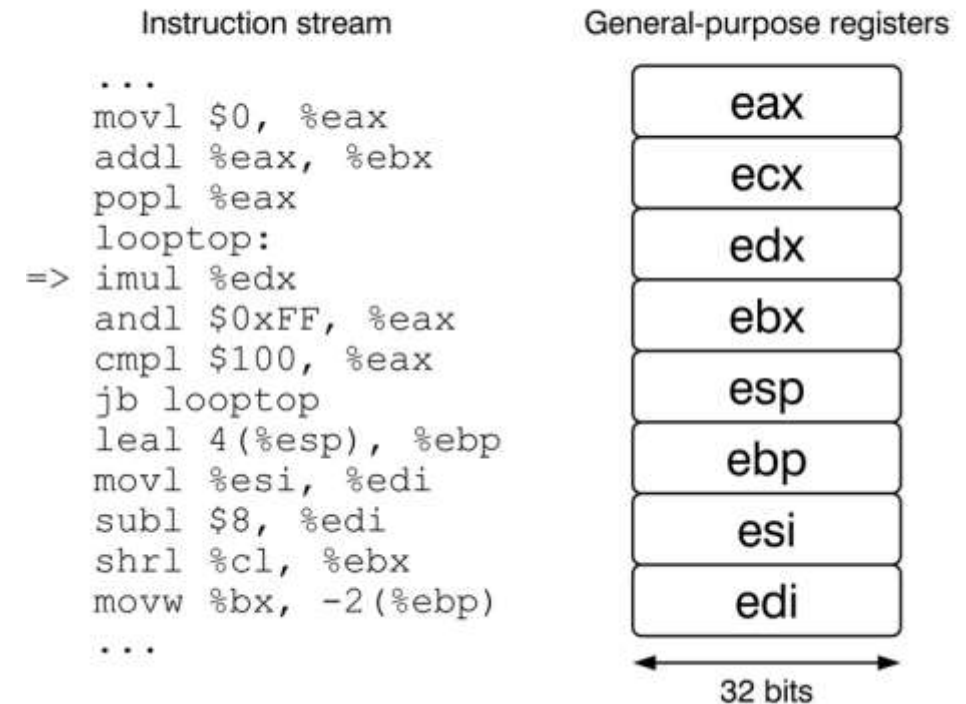
Assembly level Language

An assembly language is a low-level programming language for microprocessors – 8085, 8086

Low level Language

A Low-level language is a programming language that provides little or no abstraction of coding concepts - 1011 0011

Simplified model of x86 CPU



EXAMPLE OF JAVA

```
class HelloWorld
{
    public static void main(String args[])
    {
        System.out.println("Hello, World");
    }
}
```


EXAMPLE OF C PROGRAM

```
#include<stdio.h>
int main()
{
    print("\nHello world");
    return 0;
}
```

EXAMPLE OF C++

```
#include<iostream>
using namespace std;
int main()
{
    cout<<"\nHello world";
    return 0;
}
```

EXAMPLE OF PYTHON

```
print ("Hello world")
```

PYTHON 2.7.X VS PYTHON 3.X

Python2	Python3
Legacy: Its is still entrenched in the software at certain company	Future: It will take over Python2 by 2020
Library: Many older libraries build for Python 2 are not forwards-compatible	Library: Many of today's developers are creating libraries strictly for use with Python 3
ASCII: String are stored as ASCII by default	Unicode: Text strings are Unicode by default
$5/2 = 2$	$5/2 = 2.5$
print "hello"	print("hello")

INSTALLATION & IDE

IDE - Integrated Development Environment

An integrated development environment (IDE) is a software suite that consolidates basic tools required to write and test software.

CS50 ide – Python <replace with program name>

Anaconda – Spyder (Use play button to run)

Python – Direct Download (Enter to run)

Visual Studio Code – Best for all rounder programming

Pycharm

Netbeans

COMMENTS

They are used to main readability & turn On or Off any part of code

Rules to create comment:

for a single line comment

""" start **"""** end for a multi line comment (doc string)

KEYWORDS

Python has a set of keywords that are reserved words that cannot be used as variable names, function names, or any other identifiers. Each keyword is used to perform a specific task in python.

Examples :

False, class, finally, is, return, None, continue, for, lambda, try, True, def, from, while, and, del, global, not, with, as, elif, if, or, yield, assert, else, import, pass, break, except, in, raise.

VARIABLE

A data item that may take on more than one value during the runtime of a program.

Rules:

- Starts with Alphabets or underscore (a-z, A-Z, _)
- Then Can be a combination of Alphabets, Underscore & Number
- Cannot be Keywords

DATA TYPES

Integer – 1, 1244, -1234

Float – 1.0, 1e10, 123.324

complex – 1+0j (j represents that its complex)

string – “Hello world”, “a”, ‘a’

boolean – True, False

PRINT() - *FUNCTION*

The `print()` *function* prints the specified message to the screen, or other standard output device. The message can be a string, or any other object, the object will be converted into a string before written to the screen.

ESCAPE SEQUENCES

They are used to modify the outputs in command line or to print reserved keys.

`\` - for newline in editor

`\\` - prints `\`

`\'` - prints `'`

`\"` - prints `"`

`\n` - prints a new line

`\t` - prints a tab

TRIO : VALUE, ID, TYPE

Each variables has three types a value, id and its type. We can access them by using built in *function* named **id()**, **type()**. Normally value is accessed easily but to know its id and type these functions are used.

TYPE CASTING

There may be times when you want to specify a type on to a variable. This can be done with casting. In simplified terms **one data type can be converted to another data type.**

function for Type casing

- int()
- float()
- str()
- bool()
- list()
- tuple()
- set()

PRINT() & INPUT()

print() func has by default newline and and white space in it
input() func can also be used to print string and get input

INPUT() - *FUNCTION*

Allowing user input, Input always recognizes the user input word as **string**. Type casting is used to convert the user given input to desired data type.

DEL KEYWORD

The del keyword is used to delete objects. In Python everything is an object, so the del keyword can also be used to delete variables, lists, or parts of a list etc.

OPERATORS

Operator Types	Operands
Arithmetic operators	+, -, *, / ,** ,//,%
Assignment operators	=, +=, -=, /=, *= , **=, //=, %=
Comparison operators	==, !=, >, <, <=, >=
Logical operators	and, or, not
Identity operators	is, is not
Membership operators	in, not in

LOGICAL OPERATOR

and table - multiplication		
True	True	True
True	False	False
False	True	False
False	False	False

or table - addition		
True	True	True
True	False	True
False	True	True
False	False	False

not table - inverse	
not True	False
not False	True

PYTHON COLLECTION

- **List** is a collection which is ordered and changeable. Allows duplicate members. Denoted by `[]`
- **Tuple** is a collection which is ordered and unchangeable. Allows duplicate members. Denoted by `()`
- **Set** is a collection which is unordered and unindexed. No duplicate members. Denoted by `{}`
- **Dictionary** is a collection which is unordered, changeable and indexed. No duplicate keys. Denoted by `{key: value}`

note : Python nested collections is also possible

MUTABLE VS IMMUTABLE OBJECTS

Objects of built-in types like *(int, float, bool, str, tuple)* are immutable.

Objects of built-in types like *(list, set, dict)* are mutable.

Immutable

- Same data stores in Same location
- Different Data stores in Different location

Mutable

- Same data stores in Different location
- Different Data stores in Different location

IF ... ELSE

Python uses If else to choose between choices, its called as Decision statement

Keywords

if

else

elif

WHILE LOOP

With the while loop we can execute a set of statements as long as a condition is true.

Keywords

while

break – *stop the loop*

continue – go back to the top of my loop

FOR LOOP

For loop is used for iterating over a sequence (that is either a list, a tuple, a dictionary, a set, or a string). With the for loop we can execute a set of statements, once for each item in a list, tuple, set etc.

Keyword

for

Note: range() – is a python standard function used for numbers

FUNCTION

A function/method is a block of code which only runs when it is called. You can pass data, known as parameter/arguments into a function. A function can return data as a result.

Keyword

def

return

ARBITRARY ARGUMENTS

If you do not know how many arguments that will be passed into your function, add * before the parameter name in the function definition. This way the function will receive a tuple of arguments, and can access the items accordingly

Types of arguments

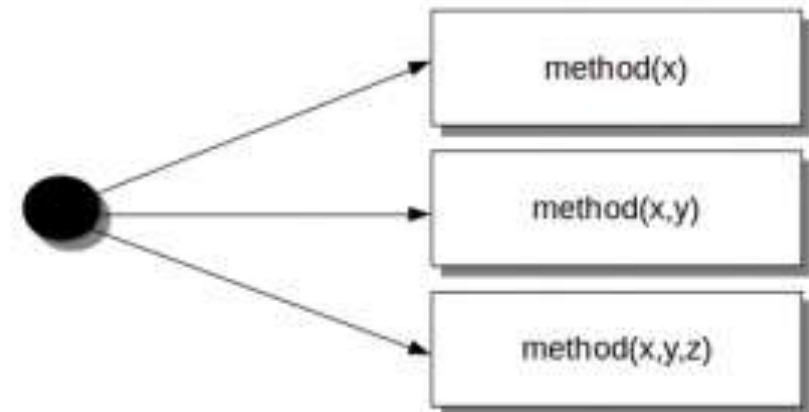
***args** - *argument stores in tuple*

****kwargs** - *keyworded argument stores in dictionary*

FUNCTION OVERLOADING

Function overloading or method overloading is the ability to create multiple functions of the same name with different implementations.

In Python you can define a method in such a way that there are multiple ways to call it. Given a single method or function, we can specify the number of parameters yourself. Depending on the function definition, it can be called with zero, one, two or more parameters.



STRING

Strings are represented in the form of arrays in Python they are either represented by “ ” or ‘ ’

str = "HELLO"

H	E	L	L	O
0	1	2	3	4

str[0] = 'H' str[:] = 'HELLO'

str[1] = 'E' str[0:] = 'HELLO'

str[2] = 'L' str[:5] = 'HELLO'

str[3] = 'L' str[:3] = 'HEL'

str[4] = 'O' str[0:2] = 'HE'

str[1:4] = 'ELL'

STRING FUNCTIONS

String Function	Description
isalpha()	Returns True if all characters in the string are in the alphabet
isdigit()	Returns True if all characters in the string are digits
isalnum()	Returns True if all characters in the string are alphanumeric
islower()	Returns True if all characters in the string are lower case
isupper()	Returns True if all characters in the string are upper case
lower()	Converts a string into lower case
upper()	Converts a string into upper case
title()	Converts the first character of each word to uppercase
swapcase()	Swaps cases, lower case becomes upper case and vice versa
split()	Splits the string at the specified separator, and returns a list
join()	Joins the elements of an iterable to the end of the string
replace()	Returns a string where a specified value is replaced with a specified value
count()	Returns the no of occurrences
find() / index()	Searches the string for a specified value and returns the position of where it was found
format()	Formats specified values in a string

STRING FORMATTING

%d – integer

%s – string

%f – float with default 6 decimal places

%.2f – float with two decimal places

%.10f – float with ten decimal places

LIST FUNCTIONS

List Function	Description
<code>append()</code>	Adds an element at the end of the list
<code>pop()</code>	Removes the element at the end of the list
<code>insert()</code>	Adds an element at the specified position
<code>remove()</code>	Removes the item with the specified value
<code>clear()</code>	Removes all the elements from the list
<code>index()</code>	Returns the index of the first element with the specified value
<code>count()</code>	Returns the number of elements with the specified value
<code>sort()</code>	Sorts the list
<code>reverse()</code>	Reverses the order of the list
<code>extend()</code>	Add the elements of a list (or any iterable), to the end of the current list
<code>copy()</code>	Returns a copy of the list

TUPLE FUNCTIONS

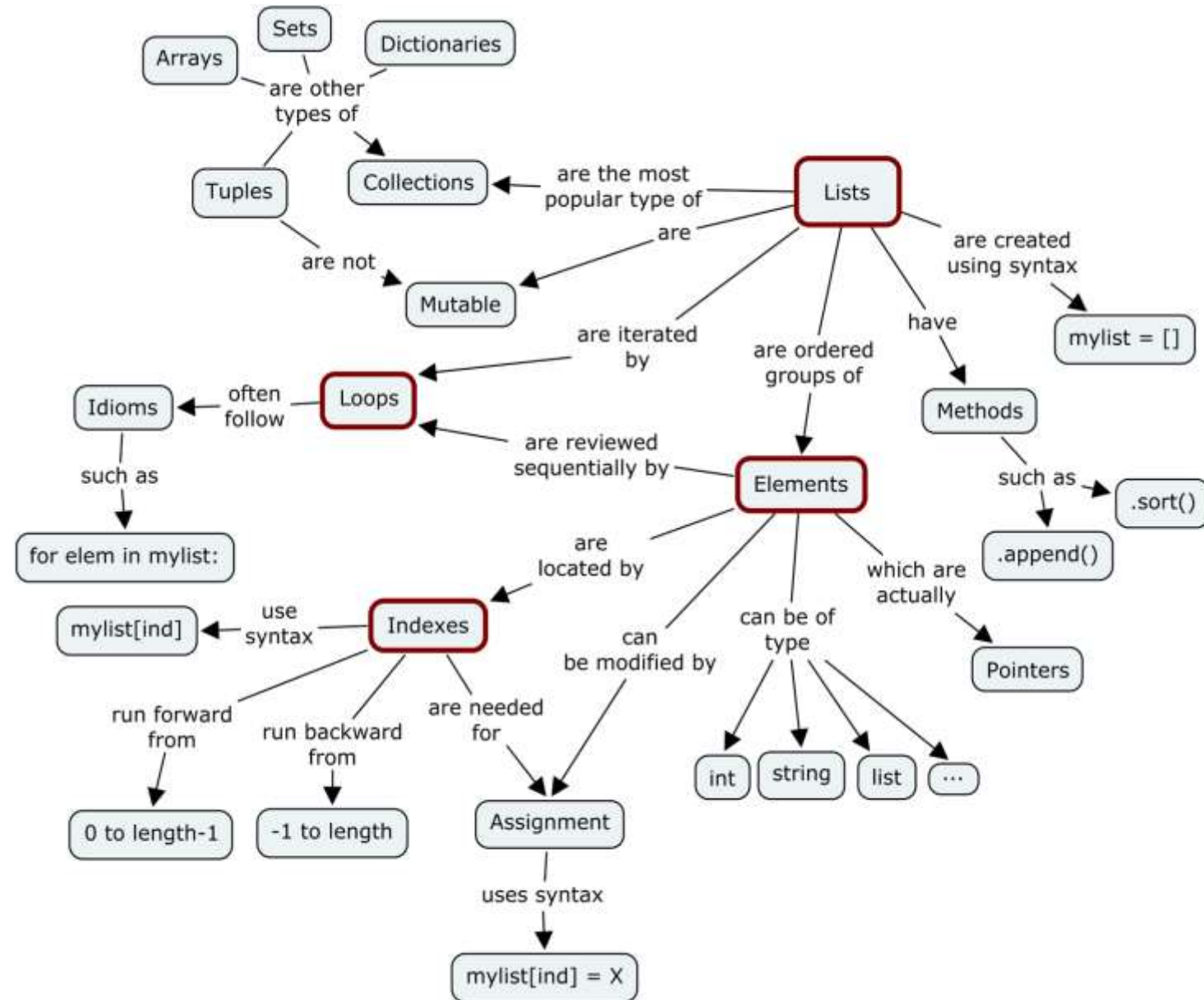
Tuple Function	Description
count()	Returns the number of elements with the specified value
index()	Returns the index of the first element with the specified value

SET FUNCTIONS

Set Function	Description
<code>union()</code>	Return a set containing the union of sets
<code>intersection()</code>	Returns a set, that is the intersection of two other sets
<code>difference()</code>	Returns a set containing the difference between two or more sets

DICTIONARY FUNCTIONS

Dictionary Function	Description
<code>keys()</code>	Returns a list containing the dictionary's keys
<code>values()</code>	Returns a list of all the values in the dictionary
<code>items()</code>	Returns a list containing a tuple for each key value pair
<code>pop(key)</code>	Removes the element with the specified key
<code>popitem()</code>	Removes the last inserted key-value pair
<code>clear()</code>	Removes all the elements from the dictionary
<code>update()</code>	Updates the dictionary with the specified key-value pairs
<code>get()</code>	Returns the value of the specified key
<code>copy()</code>	Returns a copy of the dictionary



PYTHON STANDARD FUNCTIONS

The Python interpreter has a number of functions and types built into it that are always available

print()	input()	int()	float()	str()
bool()	complex()	list()	tuple()	dict()
id()	type()	range()	len()	max()
min()	abs()	pow()	round()	sum()
slice()	sorted()	map()	filter()	zip()
eval()	exec()	chr()	ord()	enumerate()
help()	open()	super()		

ENUMERATE() - *FUNCTION*

enumerate() method adds a counter to an iterable and returns it in a form of enumerate object. This enumerate object can then be used directly in for loops or be converted into a list of tuples using list() method.

LAMBDA FUNCTION

- A lambda function is a small anonymous function.
- A lambda function can take any number of arguments, but can only have one expression.

Syntax:

funcname = lambda arguments : expression

GENERATOR FUNCTION

A generator-function is defined like a normal function, but whenever it needs to generate a value, it does so with the `yield` keyword rather than `return`. If the body of a `def` contains **`yield`**, the function automatically becomes a generator function.

MODULES & LIBRARY

import modules

Reusability of code is the main advantage of any programming language. It helps to minimize the need for writing code for the same problem when it is already solved.

import keyword is used to import from other modules. as is used to give nicknames

Example:

math , random , statistics , os , sys , date, plotly , pyautogui , calendar ,
regular expression

EXCEPTION HANDLING

Try ... Except ... finally

- The **try** block lets you test a block of code for errors.
- The **except** block lets you handle the error.
- The **finally** block lets you execute code, regardless of the result of the try and except blocks.
- The **else** block works only if *try* is correct

RAISE - *KEYWORD*

- The raise keyword is used to raise an exception.
- You can define what kind of error to raise, and the text to print to the user.

ASSERT - *KEYWORD*

Python assert statement is a debugging aid that tests a condition. If the condition is true, it does nothing and your program just continues to execute. But if the assert condition evaluates to false, it raises an `AssertionError` exception with an optional error message.

PIP – PYTHON INSTALLATION PACKAGE

`python --version` (to check the version)

`pip --version` (to check the version)

`pip list` (to check the list of all packages)

`pip freeze`

`pip install camelcase` (to install 3rd party module)

`pip uninstall camelcase` (to uninstall it)

SCOPE

Scope refers to the coding region from which particular Python object is accessible. Hence one cannot access any particular object from anywhere from the code, the accessing has to be allowed by the scope of the object.

Types of scope

- Local scope

- Global scope

RECURSION

A function is called within the same function is called Recursion.

Recursion is a common mathematical and programming concept. It means that a function calls itself. This has the benefit of meaning that you can loop through data to reach a result.

The developer should be very careful with recursion as it can be quite easy to slip into writing a function which never terminates, or one that uses excess amounts of memory or processor power. However, when written correctly recursion can be a very efficient and mathematically-elegant approach to programming.

REGULAR EXPRESSION

- A RegEx, or Regular Expression, is a sequence of characters that forms a search pattern.
- RegEx can be used to check if a string contains the specified search pattern.
- Python has a built-in package called **re**, which can be used to work with Regular Expressions

REGEX : META CHARACTERS

Character	Description	Example
[]	A set of characters	"[a-m]"
\	Signals a special sequence (can also be used to escape special characters)	"\d"
.	Any character (except newline character)	"he..o"
^	Starts with	"^hello"
\$	Ends with	"world\$"
*	Zero or more occurrences	"aix*"
+	One or more occurrences	"aix+"
?	Zero or One occurrences	"aix?"
{}	Exactly the specified number of occurrences	"al{2}"
	Either or	"falls stays"
()	Capture and group	"a(ix)+"

REGEX : SETS

Set	Description
[arn]	Returns a match where one of the specified characters (a or r or n) are present
[a-n]	Returns a match for any lower case character, alphabetically between a and n
[^arn]	Returns a match for any character EXCEPT a, r, and n
[0123]	Returns a match where any of the specified digits (0, 1, 2, or 3) are present
[0-9]	Returns a match for any digit between 0 and 9
[0-5][0-9]	Returns a match for any two-digit numbers from 00 and 59
[a-zA-Z]	Returns a match for any character alphabetically between a and z, lower case OR upper case
[+]	In sets, +, *, ., , (), \$,{} has no special meaning, so [+] means: return a match for any + character in the string

REGEX : SPECIAL SEQUENCES

Character	Description	Example
<code>\A</code>	Returns a match if the specified characters are at the beginning of the string	<code>"\AThe"</code>
<code>\b</code>	Returns a match where the specified characters are at the beginning or at the end of a word (the "r" in the beginning is making sure that the string is being treated as a "raw string")	<code>r"\bain"</code> <code>r"ain\b"</code>
<code>\B</code>	Returns a match where the specified characters are present, but NOT at the beginning (or at the end) of a word (the "r" in the beginning is making sure that the string is being treated as a "raw string")	<code>r"\Bain"</code> <code>r"ain\B"</code>
<code>\d</code>	Returns a match where the string contains digits (numbers from 0-9)	<code>"\d"</code>
<code>\D</code>	Returns a match where the string DOES NOT contain digits	<code>"\D"</code>
<code>\s</code>	Returns a match where the string contains a white space character	<code>"\s"</code>
<code>\S</code>	Returns a match where the string DOES NOT contain a white space character	<code>"\S"</code>
<code>\w</code>	Returns a match where the string contains any word characters (characters from a to Z, digits from 0-9, and the underscore _ character)	<code>"\w"</code>
<code>\W</code>	Returns a match where the string DOES NOT contain any word characters	<code>"\W"</code>
<code>\Z</code>	Returns a match if the specified characters are at the end of the string	<code>"Spain\Z"</code>

REGEX : MATCH OBJECT

A Match Object is an object containing information about the search and the result. The Match object has properties and methods used to retrieve information about the search, and the result:

.span() returns a tuple containing the start-, and end positions of the match.

.string returns the string passed into the function

.group() returns the part of the string where there was a match

Note: If there is no match, the value None will be returned, instead of the Match Object

FILE HANDLING

File handling is an important part of any web application. Python has several functions for creating, reading, updating, and deleting files.

open() – function

- The key function for working with files in Python is the open() function.
- The open() function takes two parameters; filename, and mode.
- There are four different methods (modes) for opening a file:

TYPE OF OPENING A FILE

"r" - Read - a file for reading, error if the file does not exist

"a" - Append - Opens a file for appending, creates the file if it does not exist

"w" - Write - Opens a file for overwriting, creates the file if it does not exist

"x" - Create - Creates the Default value. Opens e specified file, returns an error if the file exists

REGEX FUNCTIONS

Function	Description
<code>findall()</code>	Returns a list containing all matches
<code>search()</code>	Returns a Match object if there is a match anywhere in the string
<code>split()</code>	Returns a list where the string has been split at each match
<code>sub()</code>	Replaces one or many matches with a string

IMPORT OS

The `os` module allows us to access functionality of the underlying operating system. So we can perform tasks such as: navigate the file system, obtain file information, rename files, search directory trees, fetch environment variables, and many other operations.

IMPORT DATETIME

A date in Python is not a data type of its own, but we can import a module named datetime to work with dates as date objects.

Directive	Description	Example
%a	Weekday, short version	Wed
%A	Weekday, full version	Wednesday
%w	Weekday as a number 0-6, 0 is Sunday	3
%d	Day of month 01-31	31
%b	Month name, short version	Dec
%B	Month name, full version	December
%m	Month as a number 01-12	12
%y	Year, short version, without century	18
%Y	Year, full version	2018
%H	Hour 00-23	17
%I	Hour 00-12	05
%p	AM/PM	PM
%M	Minute 00-59	41
%S	Second 00-59	08
%f	Microsecond 000000-999999	548513
%j	Day number of year 001-366	365
%U	Week number of year, Sunday as the first day of week, 00-53	52
%W	Week number of year, Monday as the first day of week, 00-53	52
%c	Local version of date and time	Mon Dec 31 17:41:00 2018
%x	Local version of date	12/31/18
%X	Local version of time	17:41:00

PYTHON JSON

- JSON is a syntax for storing and exchanging data.
- JSON is text, written with JavaScript object notation.
- Python has a built-in package called json, which can be used to work with JSON data.
- Converting Python Object to Json using `json.dumps()`
- Converting Json to Python Object using `json.loads()`

PYTHON TO JSON

Python	JSON
dict	Object
List	Array
Tuple	Array
str	String
int	Number
float	Number
True	True
False	False
None	null

IMPORT CSV

The csv module implements classes to read and write tabular data in CSV format. It allows programmers to say, “write this data in the format preferred by Excel,” or “read data from this file which was generated by Excel,” without knowing the precise details of the CSV format used by Excel.

CLASSES & OBJECTS

Python is an object oriented programming language. Almost everything in Python is an object, with its properties and methods. A Class is like an object constructor, or a "blueprint" for creating objects.

Objects are instance (copy) of a class that can be created and destroyed later

Keywords

class

def

SELF PARAMETER

The self parameter is a reference to the current instance of the class, and is used to access variables that belongs to the class. It does not have to be named self , you can call it whatever you like, but it has to be the first parameter of any function in the class:

*note : function inside class should have **self** as its first argument*

CONSTRUCTOR - *FUNCTION*

A constructor is a special kind of function/method that Python calls when it instantiates an object using the definitions found in your class. Python relies on the constructor to perform tasks such as initializing (assigning values to) any instance variables that the object will need when it starts.

Syntax:

`__init__()`

DESTRUCTORS - *FUNCTION*

Destructors are called when an object gets destroyed. In Python, destructors are not needed as much needed in C++ because Python has a garbage collector that handles memory management automatically.

Syntax:

`__del__()`

OOPS CONCEPTS

-(OBJECT-ORIENTED
PROGRAMMING)

- Object
- Class
- Method/function
- Polymorphism
- Encapsulation
- Data Abstraction
- Inheritance

POLYMORPHISM

The word polymorphism means having many forms. In simple words, we can define polymorphism as the ability of a message to be displayed in more than one form.

Real life example of polymorphism, a person at the same time can have different characteristics. Like a man at the same time is a father, a husband, an employee. So the same person possesses different behavior in different situations. This is called polymorphism.

ENCAPSULATION

In normal terms Encapsulation is defined as wrapping up of data and information under a single unit. In Object Oriented Programming, Encapsulation is defined as binding together the data and the functions that manipulates them.

DATA ABSTRACTION

Data Abstraction - Abstraction means displaying only essential information and hiding the details. Data abstraction refers to providing only essential information about the data to the outside world, hiding the background details or implementation.

INHERITANCE

Inheritance allows us to define a class that inherits all the methods and properties from another class. Parent class is the class being inherited from, also called base class. Child class is the class that inherits from another class, also called derived class.

Single inheritance: When a child class inherits from only one parent class, it is called as single inheritance.

Multiple inheritance: When a child class inherits from multiple parent classes, it is called as multiple inheritance.

Multilevel inheritance: When we have child and grandchild relationship.

Hierarchical inheritance: More than one derived classes are created from a single base.

Hybrid inheritance : Formed by combining the above types of inheritance

MRO - METHOD RESOLUTION ORDER

Method resolution order defines the order in which the base classes are searched when executing a method. First, the method or attribute is searched within a class and then it follows the order we specified while inheriting. This order is also called Linearization of a class and set of rules are called MRO(Method Resolution Order)

SUPER() - *FUNCTION*

The `super()` builtin returns a proxy object (temporary object of the superclass) that allows us to access methods of the base class.

In Python, `super()` has two major use cases:

- Allows us to avoid using the base class name explicitly
- Working with Multiple Inheritance

SPECIAL VARIABLE/ MAGIC METHOD

Python comes with a number of special variables and methods whose name is preceded and followed by `__`. In fact, because of this, it's recommended that you don't write your own variables using this pattern as to not confuse the Python interpreter.

Examples:

`__init__()`

`__str__()`

`__name__`

`__main__`

OPERATOR OVERLOADING

Operator Overloading means giving extended meaning beyond their predefined operational meaning.

For example operator + is used to add two integers as well as join two strings and merge two lists. It is achievable because '+' operator is overloaded by int class and str class. You might have noticed that the same built-in operator or function shows different behavior for objects of different classes, this is called Operator Overloading.

OPERATOR OVERLOADING MAGIC METHODS

Binary Operator

OPERATOR	MAGIC METHOD
+	<code>__add__(self, other)</code>
-	<code>__sub__(self, other)</code>
*	<code>__mul__(self, other)</code>
/	<code>__truediv__(self, other)</code>
//	<code>__floordiv__(self, other)</code>
%	<code>__mod__(self, other)</code>
**	<code>__pow__(self, other)</code>

Comparison Operator

OPERATOR	MAGIC METHOD
<	<code>__lt__(self, other)</code>
>	<code>__gt__(self, other)</code>
<=	<code>__le__(self, other)</code>
>=	<code>__ge__(self, other)</code>
==	<code>__eq__(self, other)</code>
!=	<code>__ne__(self, other)</code>

OPERATOR OVERLOADING MAGIC METHODS

Assignment Operators

OPERATOR	MAGIC METHOD
+=	__iadd__(self, other)
-=	__isub__(self, other)
*=	__imul__(self, other)
/=	__idiv__(self, other)
//=	__ifloordiv__(self, other)
%=	__imod__(self, other)
**=	__ipow__(self, other)